

PARTIE 3 ANIMATIONS ET DÉFIS GUIDÉS

Document distribué uniquement au début de la séance

Deux tâches accessibles

Tous les étudiants créent une animation simple, puis l'enseignant attribue un défi parmi les quatre. L'extension demeure facultative.

Démarrage obligatoire

- Ouvrir le projet et lancer la scène.
- Corriger uniquement les erreurs bloquantes.
- Effectuer le commit : classe: enregistrer l'état initial du prototype.
- Inscrire le défi attribué ci-dessous.

Défi attribué : 1 / 2 / 3 / 4

Nom : Kevin Emmanuel Jean Projet : Mini-Jeux2D

Règles communes

- Suivre les étapes dans l'ordre.
- Tester après chaque étape importante.
- Conserver les fonctionnalités déjà valides.
- Effectuer au moins trois commits pendant la séance.
- Demander une validation avant de commencer l'extension.

Composants utiles

Besoin	Composants possibles
Créer une animation	Animation, Animator et clips
Détecter un contact	Collider2D et Is Trigger
Déplacer un objet	Transform ou Rigidbody2D
Afficher une information	Canvas et composant texte
Configurer une valeur	Variable public ou SerializeField

ÉTAPE COMMUNE CRÉER LES ANIMATIONS

Résultat attendu : le personnage joue Idle lorsqu'il est immobile et Move lorsqu'il se déplace.

1 Préparer les sprites

- Créer les dossiers Animations et Animator.
- Sélectionner la feuille de sprites.
- Choisir Sprite Mode Multiple.
- Ouvrir Sprite Editor, effectuer le découpage puis Apply.

2 Créer les clips

- Sélectionner les images immobiles et créer Idle.anim.
- Sélectionner les images de déplacement et créer Move.anim.
- Vérifier Sample Rate et activer Loop Time pour les deux clips.

3 Configurer Animator

- Ouvrir la fenêtre Animator.
- Créer un paramètre Float nommé Vitesse.
- Créer Idle vers Move avec Vitesse Greater 0.01.
- Créer Move vers Idle avec Vitesse Less 0.01.
- Désactiver Has Exit Time pour les deux transitions.

4 Relier le script

```
using UnityEngine;

public class AnimationJoueur : MonoBehaviour
{
    private Animator animator;

    private void Awake()
    {
        animator = GetComponent<Animator>();
    }

    public void MettreAJour(Vector2 direction)
    {
        animator.SetFloat("Vitesse", direction.sqrMagnitude);
    }
}
```

Dans le script de déplacement

```
// Après avoir calculé le mouvement :
animationJoueur.MettreAJour(mouvement);
```

Solution de remplacement

Si le personnage ne possède pas de feuille de sprites, créer une animation simple dans la fenêtre Animation en faisant varier légèrement son échelle ou sa rotation.

Tests

- Idle joue sans entrée clavier.
- Move joue pendant le déplacement.
- Les transitions ne restent pas bloquées.

- Les clips jouent en boucle.

DÉFI 1 Objet à collecter et compteur

Résultat attendu : Le compteur augmente et l'objet disparaît lorsque le joueur le collecte.

Étapes

- Créer un objet correspondant au thème.
- Ajouter un Collider2D et cocher Is Trigger.
- Ajouter le script à l'objet.
- Assigner ou retrouver le texte du compteur.
- Compléter les lignes TODO.
- Placer plusieurs copies de l'objet et tester.

Script de départ

```
using UnityEngine;

public class ObjetCollecte : MonoBehaviour
{
    private static int compteur = 0;

    private void OnTriggerEnter2D(Collider2D autre)
    {
        // TODO 1 : vérifier que autre possède le tag Player
        // TODO 2 : augmenter compteur
        // TODO 3 : afficher la nouvelle valeur
        // TODO 4 : détruire cet objet
    }
}
```

Tests

- Le contact avec un autre objet ne déclenche rien.
- Le compteur augmente une seule fois.
- L'objet collecté disparaît.

Extension facultative

Afficher le compteur dans le Canvas ou produire un petit effet visuel lors de la collecte.

Commits suggérés

- feat: préparer les objets du défi
- feat: programmer le comportement principal
- fix: tester et corriger le défi

DÉFI 2 Zone dangereuse et retour

Résultat attendu : Le joueur revient à sa position initiale et reçoit une rétroaction visible.

Étapes

- Créer une zone et lui ajouter un Collider2D en mode Trigger.
- Créer le script ZoneDangereuse.
- Enregistrer la position initiale du joueur.
- Détecter le joueur lorsqu'il entre dans la zone.
- Replacer le joueur à la position enregistrée.
- Ajouter un message ou un bref changement de couleur.

Script de départ

```
using UnityEngine;

public class ZoneDangereuse : MonoBehaviour
{
    private Vector3 positionDepart;

    private void Start()
    {
        // TODO 1 : trouver le joueur
        // TODO 2 : enregistrer sa position
    }

    private void OnTriggerEnter2D(Collider2D autre)
    {
        // TODO 3 : vérifier le tag Player
        // TODO 4 : remplacer le joueur
        // TODO 5 : afficher un message
    }
}
```

Tests

- Le joueur peut entrer dans la zone.
- Il revient à la bonne position.
- La zone ne déplace pas les autres objets.
- La rétroaction est visible.

Extension facultative

Ajouter une seconde position de retour configurable dans l'Inspector.

Commits suggérés

- feat: préparer les objets du défi
- feat: programmer le comportement principal
- fix: tester et corriger le défi

DÉFI 3 Ennemi en patrouille simple

Résultat attendu : L'ennemi se déplace continuellement entre deux points.

Étapes

- Créer deux objets vides PointA et PointB.
- Placer un ennemi entre les deux points.
- Ajouter le script PatrouilleSimple.
- Glisser les deux points dans l'Inspector.
- Compléter le déplacement et le changement de cible.
- Ajouter une conséquence simple au contact du joueur.

Script de départ

```
using UnityEngine;

public class PatrouilleSimple : MonoBehaviour
{
    [SerializeField] private Transform pointA;
    [SerializeField] private Transform pointB;
    [SerializeField] private float vitesse = 2f;
    private Transform cible;

    private void Start()
    {
        // TODO 1 : choisir pointB comme première cible
    }

    private void Update()
    {
        // TODO 2 : avancer vers cible
        // TODO 3 : si la cible est atteinte, changer de cible
    }
}
```

Tests

- Les deux références sont assignées.
- L'ennemi atteint les deux points.
- La vitesse peut être changée dans l'Inspector.
- Le contact avec le joueur produit l'effet prévu.

Extension facultative

Retourner visuellement le sprite lorsque la direction change.

Commits suggérés

- feat: préparer les objets du défi
- feat: programmer le comportement principal
- fix: tester et corriger le défi

DÉFI 4 Porte conditionnelle

Résultat attendu : La porte devient accessible uniquement lorsque l'objectif est atteint.

Étapes

- Choisir une condition déjà présente : collecte, score ou objectif.
- Placer la porte ou la zone de sortie.
- Créer le script PorteConditionnelle.
- Définir la quantité nécessaire dans l'Inspector.
- Afficher un message si la condition est incomplète.
- Permettre la progression lorsque la condition est satisfaite.

Script de départ

```
using UnityEngine;

public class PorteConditionnelle : MonoBehaviour
{
    [SerializeField] private int quantiteNecessaire = 3;
    private bool ouverte = false;

    public void VerifierCondition(int quantiteActuelle)
    {
        // TODO 1 : comparer les deux quantités
        // TODO 2 : mettre ouverte à true si nécessaire
        // TODO 3 : changer l'apparence de la porte
    }

    private void OnTriggerEnter2D(Collider2D autre)
    {
        // TODO 4 : vérifier le joueur et l'état ouverte
        // TODO 5 : afficher le message approprié
    }
}
```

Tests

- La porte est inaccessible au départ.
- Le message indique ce qui manque.
- La porte change lorsque la condition est atteinte.
- La progression fonctionne seulement lorsque ouverte vaut true.

Extension facultative

Ajouter un son ou une petite animation au moment du déverrouillage.

Commits suggérés

- feat: préparer les objets du défi
- feat: programmer le comportement principal
- fix: tester et corriger le défi

VALIDATION ET REMISE

Liste de contrôle

Vérification	Oui
Les clips Idle et Move existent.	
Animator change correctement d'état.	
Le défi principal fonctionne.	
La fonctionnalité est intégrée au thème.	
Une valeur peut être changée dans l'Inspector.	
Une rétroaction est visible.	
Trois commits ont été effectués.	
Le README indique les ressources utilisées.	
Le dernier push est visible sur GitHub.	

Courte explication

- Quel GameObject contient Animator? : Player
- À quoi sert le paramètre Vitesse? : Vitesse à laquelle le joueur bouge.
- Quel GameObject contient le script du défi? : GameManager
- Quelle variable contrôle le comportement? : GameManager

Modification simple demandée par l'enseignant

Modification : 1 / 2 / 3 / 4

Résultat :

Critère principal

Les animations et le défi principal doivent être fonctionnels et compris. L'extension reste facultative.